ASSIGNMENT 02 REPORT

Course: Embedded Linux Systems

Write Embedded Application - Sensor Monitoring System

TEAM MEMBERS (04)

No.	Full name	Student ID
01	Lê Tiến Thành	SE201799
02	Lê Tấn Lộc	SE201412
03	Trần Đình Khôi Nguyên	SE200558
04	Trần Trương Tuấn Vũ	SE196467

Lecturer: Phạm Thế Vinh

October 2026

Contents

1. Goals and environment
2. Architecture design
3. Thread synchronization
4. Driver and application integration
5. Memory management
6. Debugging and profiling
7. Real-time behavior and jitter
8. Summary of measurements
9. What is not verified yet
10. Looking back at the course
References
Appendix A. Commands used to produce the data
Appendix B. Mapping to the assignment requirements
Appendix C. Evidence files

1. Goals and environment

1.1 What this work contains

The work has two parts. The first is a kernel module, `sms_sensor_driver.ko`, that simulates a temperature and humidity sensor. It exposes data through `/dev/sms_sensor` and statistics through `/proc/sms_stats`. The second is the application `sms_app`, written in C with four POSIX threads: one reads the sensor, one processes samples, one handles alerts and one writes the log. The application runs on QEMU ARMv7 (vexpress-a9). An x86_64 build of the same source is used for Valgrind, strace, perf and GDB, because those tools are easier to run directly on the host.

The driver generates temperature and humidity from random numbers, and there is no real hardware behind it. The alert counts below therefore reflect the simulation, not a real environment. Section 8 explains the numbers.

All numbers in this report come from my own runs. Anything I have not measured or confirmed is listed in section 9.

1.2 Environment

Table 1. Environment

Item	Configuration
Host	Ubuntu 24.04, x86_64, GDB 15.1
Target	QEMU vexpress-a9, cortex-a9, 2 vCPUs, 512 MB RAM
Kernel	Linux 5.15.0, built by me, PREEMPT_NONE, with CONFIG_POSIX_MQUEUE and CONFIG_HIGH_RES_TIMERS enabled
Root filesystem	initramfs with busybox, `sms_app` linked statically
Compiler	arm-linux-gnueabihf-gcc for ARM, gcc for the host. Flags: `-Wall -Wextra -g -O2 -fno-omit-frame-pointer -pthread`
Analysis tools	Valgrind (`--leak-check=full --track-fds=yes`), `strace -f -c`, `perf record -F 999 -g`, GDB

1.3 Source files and command line options

The source is split by function, and each `.c` file starts with a comment giving its purpose, author and date. The Makefile has three targets: `make arm` (static ARM build), `make host` (x86 build for Valgrind and perf) and `make debug` (x86 build with `-O0` for GDB).

Table 2. Main files

File	Content
driver/sms_sensor_driver.c, sms_sensor.h	Kernel module, ioctl definitions and device path
app/sms_app.c	main: option parsing, initialization, thread creation, ordered shutdown, final report
app/sensor_thread.c	Periodic driver reads and jitter measurement
app/processor_thread.c	10-sample rolling average and anomaly detection
app/alert_thread.c	Receives alarms, prints to stderr, counts by type, forwards to the Logger

File	Content
app/logger_thread.c	Writes `/tmp/sms.log`, rotates at 1 MB, flushes every 5 seconds
app/circular_buffer.c, .h	Thread-safe ring buffer
app/app_config.h	Shared `AppContext` struct and constants

Table 3. Command line options of sms_app

Option	Meaning	Default
-i <ms>	Sampling interval of the Sensor thread	500
-t <deg C>	High temperature threshold	40.0
-h <%>	High humidity threshold	80.0
-d <sec>	Run time, 0 means run forever	60
-v	Print every sample to stdout	off
-s 0|1	0 is SCHED_OTHER, 1 is SCHED_FIFO for the Sensor thread (I added this to compare jitter)	0
-p <path>	Override the device path, used to run the x86 build against a mock file	/dev/sms_sensor
-l <path>	Override the log file path	/tmp/sms.log

2. Architecture design

2.1 Overall diagram

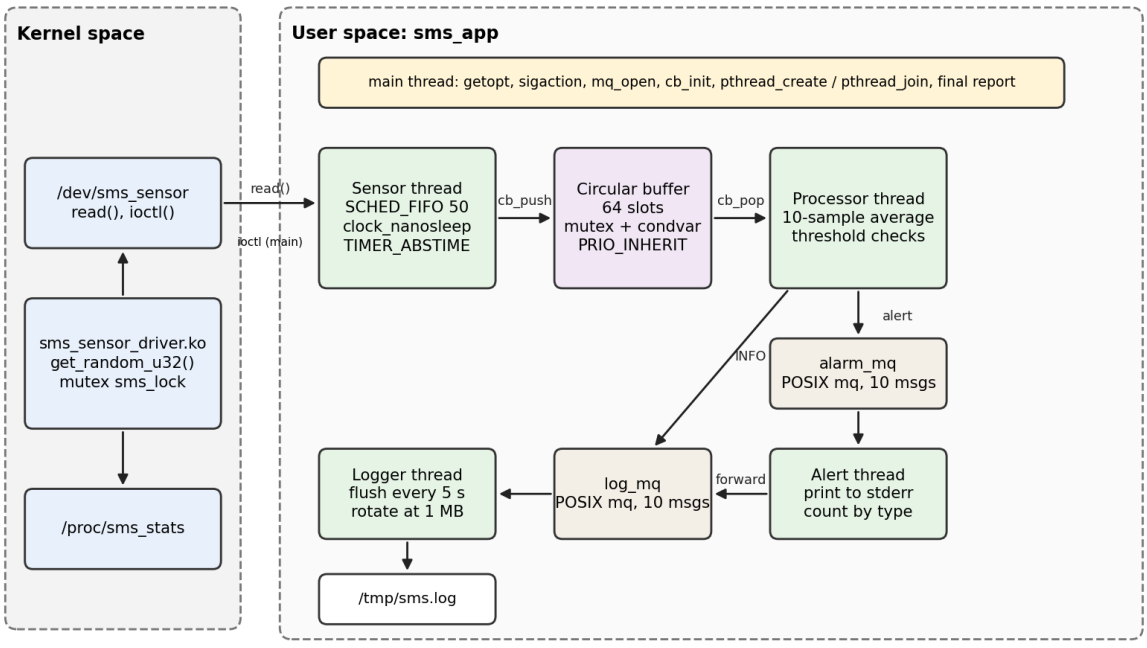


Figure 1. Architecture of the SMS system

A sample travels as follows. The Sensor thread reads one CSV line from `/dev/sms_sensor` and pushes it into the circular buffer. The Processor pops it, updates two 10-sample rolling windows (temperature and humidity), compares the values with the thresholds, then sends alert messages

to `alarm_mq` and an INFO line to `log_mq`. The Alert thread receives from `alarm_mq`, prints to stderr, counts by type and forwards the message to `log_mq`. The Logger thread is the only thread that opens `/tmp/sms.log`.

2.2 Kernel driver

The driver uses a `miscdevice` with a dynamic minor number, so I do not have to reserve a major and minor myself. On QEMU the device shows up as `crw-rw-rw- 10, 125` (Figure 3). Each `read()` returns exactly one CSV line of the form `<timestamp_ms>,<temperature>,<humidity>\n`, for example `93917,42.7,85.4`.

The kernel does not use floating point, so the driver stores values in tenths. Temperature is `150 + rand % 301` (15.0 to 45.0 degrees C) and humidity is `300 + rand % 601` (30.0 to 90.0 %), where `rand` comes from `get_random_u32()`. When printing, the integer is split with `/10` and `%10`. The timestamp is `ktime_to_ms(ktime_get())`, wrapped in a function called `sms_ktime_get_ms()`. I used this instead of `ktime_get_ms()` from the assignment text to stay compatible with kernel 5.15 on 32-bit ARM.

The driver state (`read_count`, last values, last read time, `min_interval_ms`) is protected by the mutex `sms_lock`. If two reads are closer together than `min_interval_ms`, `read()` returns `-EAGAIN`. State is updated only after `copy_to_user` succeeds, so `read_count` does not count failed reads.

Table 4. Driver interface

Interface	Behavior
read /dev/sms_sensor	One CSV line per read. Short reads are allowed if the caller buffer is smaller than the line
ioctl SMS_SET_INTERVAL	`_IOW('x', 1, __u32)`. Valid values are 1 to 60000 ms, anything else returns `-EINVAL`. Default is 100 ms
/proc/sms_stats	read_count, last_temperature, last_humidity, uptime (seconds), min_interval_ms
open / llseek	`nonseekable_open` and `no_llseek`, since this is a stream device

2.3 Threads

Table 5. Responsibilities of the four threads

Thread	Job	Waits on	Writes to
Sensor	Reads the driver, measures jitter, pushes samples. SCHED_FIFO 50 when started with `-s 1`	`clock_nanosleep` with TIMER_ABSTIME at the `-i` period	circular buffer
Processor	10-sample rolling average, checks the three alert conditions	`pthread_cond_wait` inside `cb_pop`	`alarm_mq`, `log_mq`
Alert	Prints alerts to stderr, counts by type, forwards to the Logger	`mq_timedreceive`, 100 ms timeout	stderr, `log_mq`
Logger	Writes the file with timestamps, rotates at 1 MB, flushes every 5 seconds	`mq_timedreceive`, 100 ms timeout	`/tmp/sms.log`

The three alert conditions are temperature above `-t`, humidity above `-h`, and a temperature change of more than 5.0 degrees from the previous sample (the first sample has no predecessor, so it is not compared). A message looks like `[ALERT] [<driver timestamp>] TYPE: TEMP_HIGH VALUE: 42.3 THRESHOLD: 40.0`. The Logger adds a system date and time prefix when it writes to the file.

The assignment asks the Alert thread to block on `alarm_mq`. I used `mq_timedreceive` with a 100 ms timeout instead of `mq_receive`, so the thread gets a chance to check its stop flag. The price is that the thread wakes a few times per second with nothing to do. Section 6.3 describes a way to remove that.

2.4 State machine

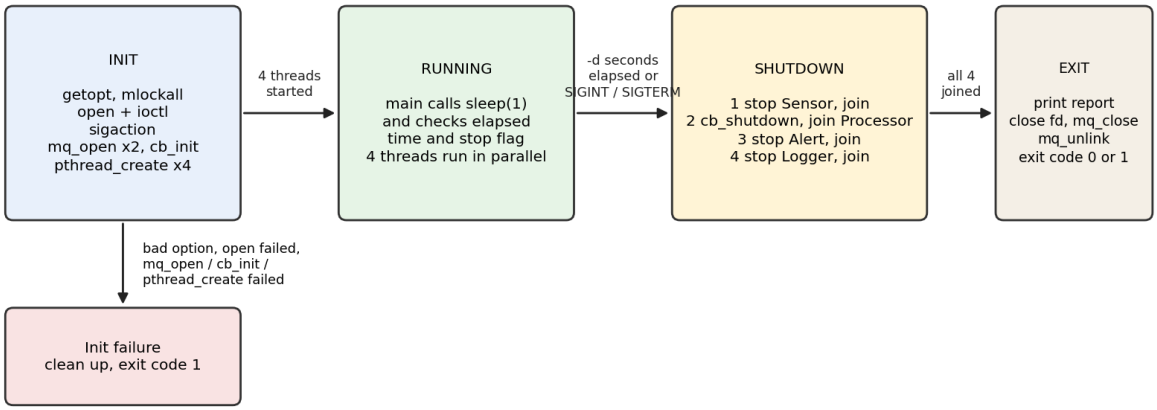


Figure 2. Application states

In INIT, if any step fails (bad option, device cannot be opened, `mq_open`, `cb_init` or `pthread_create` fails), the program releases what it has allocated and exits with code 1. In RUNNING the main thread only sleeps one second at a time and checks two conditions: `-d` seconds have passed, or SIGINT or SIGTERM arrived. Both signals only set an atomic flag, and the handler does nothing else. SHUTDOWN stops the stages in the order given in section 3.3. The final exit code is 1 if Alert or Logger wrote an error to `thread_error`, and 0 otherwise.

2.5 Design decisions

I chose a circular buffer over a linked list because the buffer is allocated once at startup. The Sensor thread then never calls `malloc` or `free` inside its time-sensitive loop, and both push and pop are O(1) without touching the heap. A `SensorData` is 16 bytes, so 64 slots use 1 KB. At a 50 ms period the buffer holds 3.2 seconds of data, which means the Processor would have to stall for more than 3 seconds before it overflows. The cost is that a full buffer drops new samples and counts them in `overflow_count`, because the buffer never grows. In every normal run `Overflows` was 0.

Alarms go through a POSIX message queue instead of shared memory because a queue already provides ordering, blocking and wake-up, timeouts, and message boundaries. With shared memory I would have to write extra synchronization for that region. Shared memory pays off when two separate processes exchange a large block of data, but here the threads share one

process and messages are under 256 bytes. The cost is one system call per send and per receive (section 6.2 counts 1500 `mq_timedsend` calls in 30 seconds).

The Logger is a separate thread because writing a file can be slow or block (opening, rotating, flushing), and I do not want the Processor or Sensor to wait for that. This design has a weakness that I ran into. In one `sudo perf` run the process ran as root and could not open `/tmp/sms.log`, because that file belonged to a normal user (`Permission denied`), so the Logger exited immediately. After that nobody read `log_mq`. Every Processor send to `log_mq` waited the full 100 ms timeout, and the buffer overflowed 74 times in 200 periods (126 samples were processed). The system did not hang, because `mq_timedsend` has a timeout, but quality dropped clearly. I record this as a failure mode worth knowing: if the Logger dies, everything in front of it slows down.

3. Thread synchronization

3.1 Shared data and protection

Table 6. Shared data

Data	Written by	Read by	Protection
Circular buffer fields (head, tail, current_size, counters)	Sensor, Processor	Processor, main	mutex `cb->lock`, all access goes through buffer functions
`cb->shutdown`	main	Processor	same mutex, plus `pthread_cond_broadcast`
Stop flag set by signals	signal handler	main	`atomic_bool`
`sensor_running`, `alert_running`, `logger_running`	main	the matching thread	`atomic_bool`
`thread_error`	Alert, Logger	main	`atomic_int`
`jitter_arr`, `jitter_count`, `fifo_active`	Sensor	main after join	single writer, main reads after `pthread_join`
Alert counters by type	Alert	main after join	same as above
`alarm_mq`, `log_mq`	Processor, Alert	Alert, Logger	the kernel protects the queues
Log `FILE *`	Logger	Logger	used by one thread only
stdout / stderr	Processor (with `-v`) and main / Alert		main prints only after all joins; only Alert writes to stderr

3.2 Lock order and why there is no deadlock

The application creates only one mutex, `cb->lock`. Every buffer function takes the lock at the start and releases it before returning. While the lock is held there is no call that can block: no `mq_send`, no printing, no second lock. So no thread holds one lock while waiting for another, hold-and-wait cannot occur, and the resource allocation graph cannot contain a cycle.

The mutex is created with `PTHREAD_PRIO_INHERIT`. The reason is that Sensor runs SCHED_FIFO and Processor runs SCHED_OTHER, yet they share the lock. If the Processor is preempted while holding it, the Sensor could be blocked for an unbounded time (priority inversion). With priority inheritance the Processor is boosted temporarily so it can release the

lock quickly. I have no separate experiment isolating the effect of this attribute, so this choice rests on theory.

Besides the mutex there are waits between threads, and the next table lists what each thread holds and waits for.

Table 7. Holds and waits per thread

Thread	Holds	Waits for
Sensor	`cb->lock` very briefly in `cb_push`	Never waits for the buffer: if it is full the sample is dropped and counted. While waiting for the next period it holds nothing
Processor	`cb->lock` inside `cb_pop`	`pthread_cond_wait` releases the lock while waiting. Sends to queues with `mq_timedsend` (100 ms), always outside the lock
Alert	nothing	`alarm_mq` (receive, 100 ms timeout), `log_mq` (send, 100 ms timeout)
Logger	nothing	`log_mq` (receive, 100 ms timeout), file writes
main	`cb->lock` briefly in getters and `cb_shutdown`	`pthread_join`

The wait relationships form a one-way chain. Sensor pushes into the buffer, Processor reads the buffer and sends into two queues, Alert reads `alarm_mq` and sends into `log_mq`, and Logger reads `log_mq` and sends to nobody. No edge points backwards. The Sensor never waits for a full buffer either, so the chain is cut at its first link. Every send into a queue also has a timeout, so if a later thread dies the earlier thread loses data but does not hang forever.

3.3 Ordered shutdown

The system stops in the same direction the data flows. Main sets `sensor_running = false` and joins the Sensor. Then it calls `cb_shutdown`. That function sets a flag and broadcasts, but `cb_pop` returns -1 only when the buffer is already empty, so the Processor handles the remaining samples before it exits. Next main stops the Alert thread and joins it, at which point nobody sends into `alarm_mq` anymore. The Alert thread leaves its main loop and then reads the rest of `alarm_mq` with a zero timeout. Finally main stops the Logger and joins it, and the Logger also reads the rest of `log_mq` before flushing and closing the file. This way the last message of each stage reaches the next stage.

3.4 Bugs found and fixed

My first version had four problems related to synchronization or IPC. All four were fixed before the measurements in this report.

The first problem was a single `keep_running` flag of type `bool`, shared by all threads, written by the signal handler and by main and read by all four threads. Under C11 that is a data race. In practice, when the flag turned off, Alert and Logger exited immediately while the Processor was still draining the buffer and sending into two queues that nobody read. The last messages were lost, and if a queue filled up (10 messages) then `mq_send` blocked forever and `pthread_join` on the Processor hung. I fixed it with three separate `atomic_bool` flags, joining in the order of section 3.3, and switching to `mq_timedsend`.

The second problem was in the Processor loop condition, which read `ctx->cb->current_size` without the lock while the Sensor was writing the same variable. I removed that condition and let the loop run on the return value of `cb_pop`. Statistics read from main now go through getter functions that take the lock.

The third problem was messages without a terminator. `mq_send` originally sent `strlen(msg)` bytes, so the received string had no `\0`, yet Alert and Logger printed it with `%s` and searched for the alert type with `strstr`. I fixed both sides: the sender now sends `strlen + 1` bytes and the receiver sets `msg[n] = '\0'` using the byte count returned by `mq_timedreceive`.

The fourth problem was `EMSGSIZE`, and it took the longest to find. Alert and Logger declared a receive buffer of `MSG_SIZE - 1` (255 bytes) while the queues were created with `mq_msgsize = 256`. According to the `mq_receive` man page the receive buffer must be at least `mq_msgsize`, otherwise every receive fails with `EMSGSIZE`. On QEMU the symptom was an ALERT SUMMARY of 0 after 263 random samples, together with `Overflows: 820`. With roughly 17 % chance of each alert type per sample, zero alerts is almost impossible. Strace on the x86 build confirmed it: `mq_timedreceive` failed 14 times out of 14 calls, and `mq_timedsend` failed 337 times out of 357 (the queue was full because nobody drained it). The fix was a `MSG_SIZE + 1` buffer and passing exactly `MSG_SIZE` to the receive call.

One more detail: alerts used to be sent with priority 1 and the INFO line with priority 0. A queue always returns the higher priority first, so the log lost its time order. I moved everything to priority 0.

4. Driver and application integration

4.1 Loading the module

The figure below shows loading the module on QEMU, checking `/dev/sms_sensor` and then running `sms_app -i 50 -d 55 -s 0`. The device has major 10 (misc) and minor 125, assigned by the kernel.

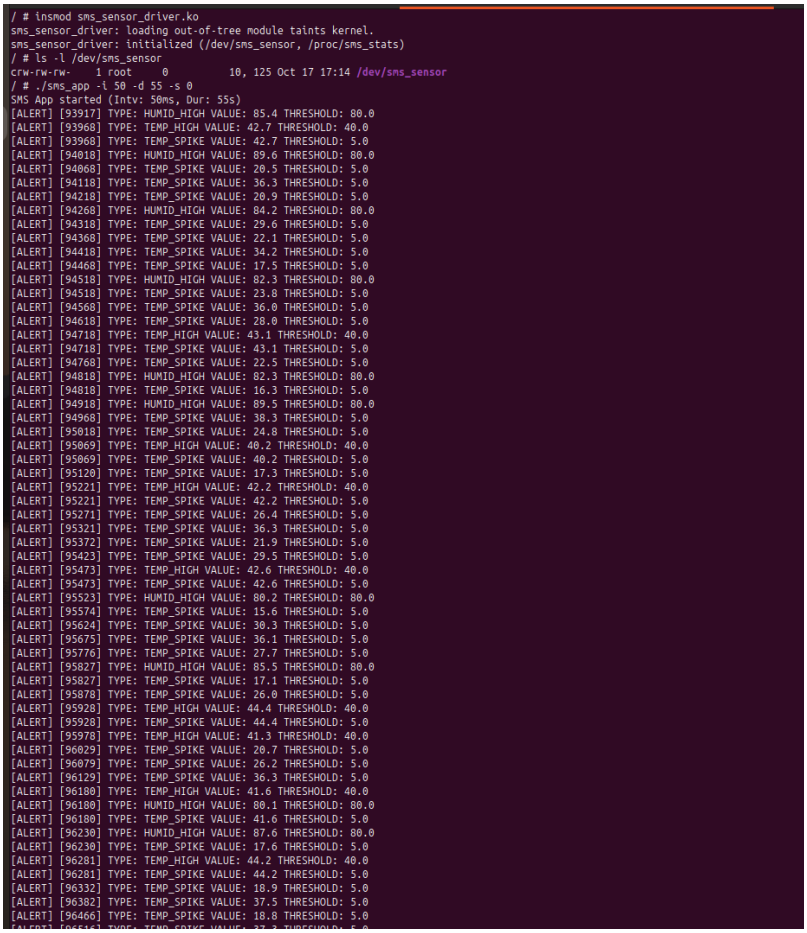


Figure 3. insmod, ls -l /dev/sms_sensor and the start of the sms_app output on QEMU

4.2 Checking against /proc/sms_stats

I ran `./sms_app -i 500 -d 60 -s 1` and read `/proc/sms_stats` before and after. The application calls `ioctl(SMS_SET_INTERVAL)` with the value of `-i`, so after the run `min_interval_ms` is 500 (it was the default 100 before).

Table 8. /proc/sms_stats before and after the 60 second run

Field	Before	After
read_count	0	118
last_temperature	25.0 C (initial value)	19.3 C
last_humidity	50.0 % (initial value)	57.8 %
uptime	0 seconds	61 seconds
min_interval_ms	100	500

60 seconds divided by 500 ms is 120 periods, and `read_count` is 118, inside the 120 plus or minus 10 range of the assignment. In the same run the application reported `Total Produced: 118`, `Dropped: 2` and 120 jitter samples. Because `read_count` equals the number of samples that reached the buffer, the two lost periods were reads that did not return data (the driver does not count them). If they had been lost at `sscanf` or `cb_push`, `read_count` would be 120.

This run is also the 60 second stability check required by Task 1. The program ran for the full 60 seconds without a crash or hang, all four stages processed data (the INFO and ALERT lines in

the log come from the Processor and the Alert thread through the Logger, and GDB listed four new threads in section 6.1), and it exited after the duration with the complete summary printed.

4.3 Handling read errors

The Sensor thread uses a file descriptor opened once in main and closed at the end of the program. When `read()` returns `EAGAIN`, `EINTR` or 0 (end of data), the thread waits 1 ms and retries, up to 10 times per period. For the return value 0 it also calls `lseek` back to the start, a branch that only matters when reading the mock file on the host. Any other error is printed with `perror` and that period is skipped. A period with no sample is counted in `Dropped`. I have not counted each errno separately, so I cannot yet say what caused the lost samples. Without load at 50 ms the loss was 6 to 17 out of 1100 periods (0.5 to 1.5 %), with load it was 0 to 2, and at 500 ms it was 2 out of 120.

5. Memory management

5.1 Valgrind

The host has no `/dev/sms_sensor`, so the x86 build runs with `-p sms_mock_sensor`, a file that holds exactly one line, `1500,42.3,85.2`. With this data the Processor, Alert and Logger all work, and every sample produces one `TEMP_HIGH` and one `HUMID_HIGH` alert. The command was:

```
valgrind --leak-check=full --show-leak-kinds=all --track-fds=yes \
  ./sms_app_x86 -p sms_mock_sensor -i 200 -d 60
```

Table 9. Valgrind results of two runs

	Run 1 (SIGINT after 30 s)	**Run 2 (Figure 4)**
Heap	19 allocs, 19 frees, 16,947 bytes	18 allocs, 18 frees, 12,699 bytes
Valgrind verdict	All heap blocks were freed, no leaks are possible	All heap blocks were freed, no leaks are possible
ERROR SUMMARY	0 errors from 0 contexts	0 errors from 0 contexts
File descriptors open at exit	4: 0, 1, 2 and fd 3, which is the log file of Valgrind itself	3: only 0, 1, 2
Samples processed		74 samples, 74 TEMP_HIGH alerts, 74 HUMID_HIGH alerts

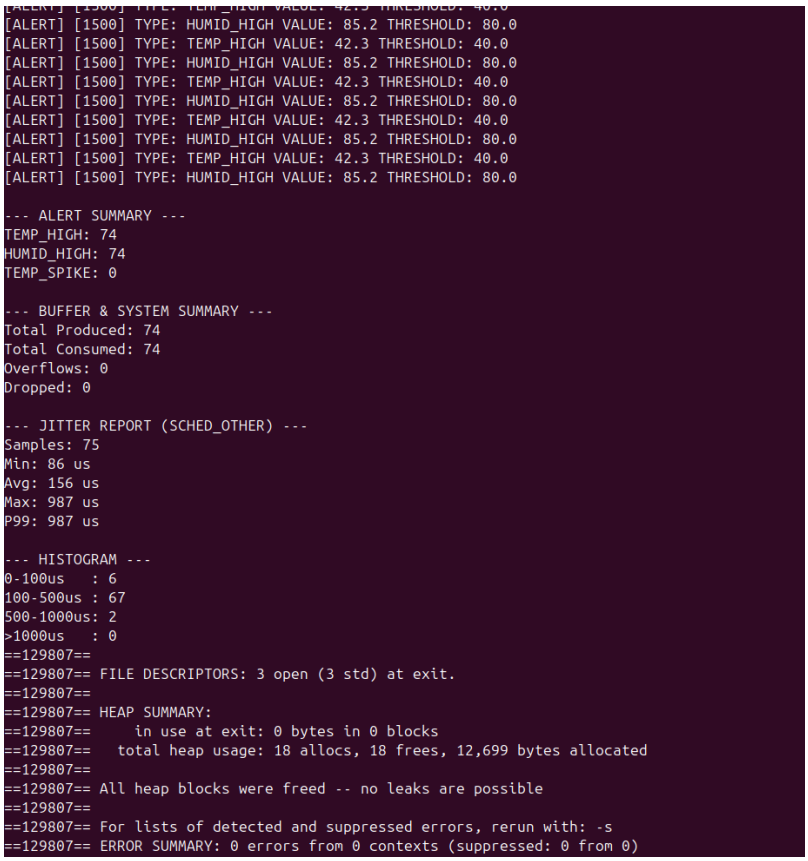


Figure 4. End of the output of the second Valgrind run

No byte is definitely lost, indirectly lost, possibly lost or still reachable, so there is nothing more to explain. For descriptors, Valgrind lists all of them at exit, and apart from the three standard ones there is none from the application (device or mock file, log, queues), so there is no fd leak.

This result has three limits. The real driver read path does not run under Valgrind because the driver exists only in the ARM guest. The mock file has a single value, so the `TEMP_SPIKE` branch never ran. And Valgrind ran in the default SCHED_OTHER mode. Run 1 was stopped with SIGINT, so the signal shutdown path was exercised too: the program printed its full report and exited cleanly.

5.2 Memory footprint on QEMU

I ran `sms_app -i 500 -d 60 -s 1` in the background and read `/proc/PID/status` at three moments.

Table 10. VmSize and VmRSS over time

Moment	VmSize	VmRSS
T = 5 s	34560 kB	33412 kB
T = 30 s	34560 kB	33412 kB
T = 55 s	34560 kB	33412 kB

Both values stay constant through the run, so heap, stacks and log buffers do not grow over time. That fits the design: the buffer is allocated once, the jitter array lives inside `AppContext` (5000 `int64_t` entries, about 40 KB), and the Logger stdio buffer has a fixed size.

What caught my attention more is that RSS is close to VmSize. The program has only about 450 KB of code but the RSS is 33 MB. The `maps` table below shows why: there are four stack regions of 8 MB each. Because `main` calls `mlockall(MCL_CURRENT | MCL_FUTURE)` before creating threads, all of these regions are locked and loaded into RAM at once instead of being filled in as they are touched. I did not read `VmLck` to confirm it, but the numbers match this explanation (4 x 8192 kB = 32768 kB, and adding a 1 MB malloc arena gets close to 34560 kB).

Table 11. Regions in /proc/PID/maps (ARM, static link)

Address	Perm	Size	Content
00010000-0007f000	r-xp	444 kB	Text: program code, including glibc since the link is static
0007f000-00082000	r--p	12 kB	Read-only data (rodata, relro)
00082000-00084000	rw-p	8 kB	Initialized data and .bss
00084000-000a8000	rw-p	144 kB	[heap] of the main thread (brk): CircularBuffer, the 1 KB buffer, glibc allocations
74d66000-75566000 and three regions of the same size	rw-p	4 x 8192 kB	Stacks of the four threads, each preceded by a 4 kB `---p` guard page
75e00000-75e21000 and 75e21000-75f00000	rw-p and ---p	132 kB and 892 kB	A separate malloc arena (1 MB reserved, 132 kB used) created by a secondary thread when it allocates, for example the Logger stdio buffer
7ef96000-7efb7000	rw-p	132 kB	[stack] of the main thread, which also holds `AppContext` with its 40 KB jitter array
7efeb000 to ffff1000	r-xp, r--p	4 kB each	[sigpage], [vvar], [vdso], [vectors] provided by the ARM kernel

The ARM build is statically linked, so there is no shared library region (libc.so). That code sits in the text region. To see shared library regions you have to look at `maps` of the dynamically linked x86 build.

6. Debugging and profiling

6.1 GDB, scenario B (debugging the alert logic)

I chose scenario B. The mock file contains `1500,24.5,50.0`, an ordinary reading that produces no alert. I started GDB with `gdb --args ./sms_app_x86 -p sms_mock_sensor -i 500 -d 15`. I set the breakpoint at `processor_thread.c:45`, and GDB moved it to line 46, the statement `if (data.temperature > ctx->temp_thresh)`.

```
Reading symbols from ./sms_app_x86...
(gdb) b processor_thread.c:45
Breakpoint 1 at 0x38f7: file processor_thread.c, line 46.
(gdb) run
Starting program: /home/letienthanh/SE_201799_LeTienThanh_Asgn02/app/sms_app_x86 -p sms_mock_sensor -i 500 -d 15

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
[New Thread 0x7ffff7bff6c0 (LWP 127447)]
[New Thread 0x7fffef1ff6c0 (LWP 127448)]
[New Thread 0x7ffff73fe6c0 (LWP 127449)]
[New Thread 0x7ffff6bfd6c0 (LWP 127450)]
SMS App started (Intv: 500ms, Dur: 15s)
[Switching to Thread 0x7ffff73fe6c0 (LWP 127449)]

Thread 4 "sms_app_x86" hit Breakpoint 1, processor_thread_func (arg=0x7fffffff3d30) at processor_thread.c:46
46              if (data.temperature > ctx->temp_thresh) {
(gdb) print data.temperature
$1 = 24.5
(gdb) set variable data.temperature=45.0
(gdb) continue
Continuing.

Thread 4 "sms_app_x86" hit Breakpoint 1, processor_thread_func (arg=0x7fffffff3d30) at processor_thread.c:46
46              if (data.temperature > ctx->temp_thresh) {
(gdb) S
```

Figure 5. GDB session: stop in the Processor, change the temperature to 45.0, continue

Four new threads are created (LWP 127447 to 127450). The breakpoint stops in "Thread 4", which is the Processor: GDB numbers main as 1, then Logger, Alert, Processor and Sensor as 2, 3, 4 and 5, matching the `pthread_create` order in `main`. At the stop, `print data.temperature` gives 24.5. I typed `set variable data.temperature=45.0` and `continue`, and GDB stopped again at the next iteration (a new sample 500 ms later).

The expectation is that 45.0 is above the 40.0 threshold, so the Processor sends `TEMP_HIGH` to `alarm_mq` and the Alert thread prints it to stderr. In the captured session no `[ALERT]` line appears between `continue` and the next stop. I have not confirmed why. The most likely explanation is that `sms_app_x86` was built with `-O2`, so the compiler may have loaded `data.temperature` into a register before line 46, and changing the value in memory does not affect the comparison. The way to check is to run again with the `make debug` build (`-O0`). The practical lesson is that variable values cannot be trusted when debugging an optimized build.

So this session shows the breakpoint, the thread numbering and the variable change, but it does not show the alert travelling through `alarm_mq` to the Alert thread, which is what scenario B asks to observe. Section 9 lists the commands for the re-run. This session also has no `info threads` or `thread apply all bt`, since those belong to scenario A.

6.2 strace

I ran `strace -f -c` (with `-f` to follow all four threads) for 30 seconds with `-i 100` and the mock file, which is 300 samples. The next table shows the top lines.

Table 12. strace -f -c, 30 seconds, 300 samples

Syscall	% time	Calls	Errors	What it maps to in the application
clock_nanosleep	37.45	630	0	300 Sensor periods, 300 waits of 1 ms when the mock file hits end of file, 30 calls of `sleep(1)` in main

Syscall	% time	Calls	Errors	What it maps to in the application
mq_timedreceive	33.18	1572	72	1500 successful receives (Alert 600, Logger 900) and 72 timeouts of 100 ms
futex	16.30	908	1	`pthread_cond_wait` and `signal` between Sensor and Processor, mutex contention, `pthread_join`
mq_timedsend	4.30	1500	0	5 sends per sample: 2 alarms, 1 INFO, 2 forwarded alerts
mprotect	3.91	8	0	Thread stack and arena setup, 4.2 ms per call (probably because of `mlockall`)
write	1.62	620	0	600 alert lines to stderr (unbuffered), the rest are the banner and the summary
read	1.12	602	0	Two reads per sample (the data line, then end of file) on the mock file
lseek	0.42	301	0	Rewind of the mock file after each end of file

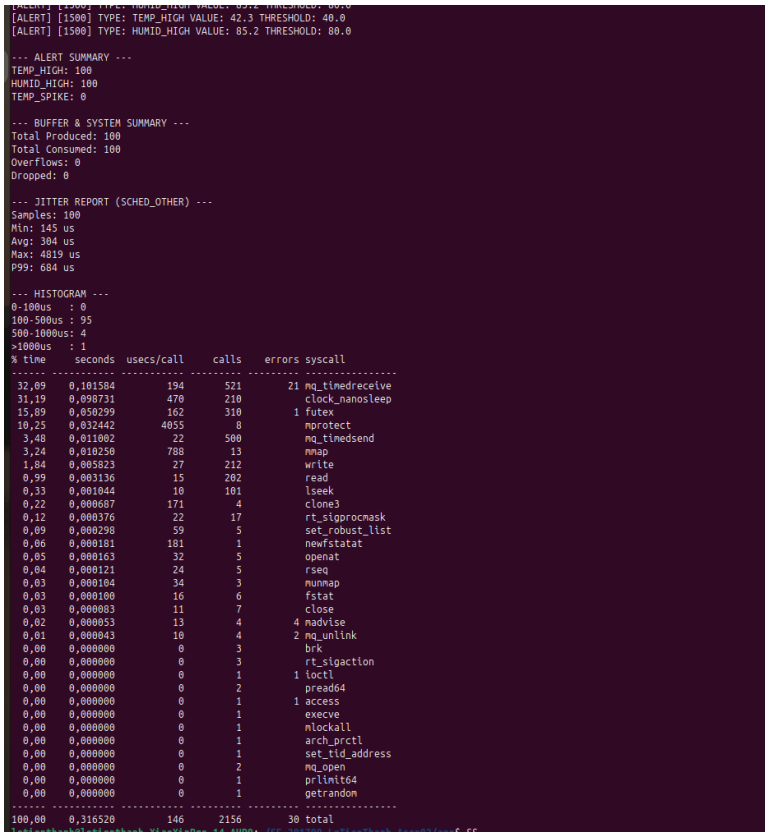


Figure 6. strace output of a 10 second run (100 samples), counted the same way

The five syscalls with the largest share of time are `clock_nanosleep`, `mq_timedreceive`, `futex`, `mq_timedsend` and `mprotect`. The counts match the code: 300 samples times 5 sends gives 1500 `mq_timedsend` calls, and exactly 1500 successful `mq_timedreceive` calls. In the 10 second run in the figure, 100 samples give 500 `mq_timedsend`, 210 `clock_nanosleep` and 212 `write`, by the same formulas. The extra `lseek`, `read` and the 300 waits of 1 ms exist only because of the mock file, and the real driver will not produce them.

On the user versus kernel split: `strace -c` counts CPU time spent inside the kernel for each system call. Adding that column gives about 0.86 seconds over 30 seconds. I did not measure user-mode time with strace. Together with perf in the next section (about 0.73 seconds of CPU in

20 seconds), the application uses under 5 % of one core, so it is not CPU-bound. Most of the time it waits for timers and queues. If I have to pick between the two labels it is closer to I/O-bound, although in truth it is mostly idle.

6.3 perf

The command was `sudo perf record -F 999 -g ./sms_app_x86 -p sms_mock_sensor -l /tmp/sms_perf.log -i 10 -d 20`, where `-l` sets a separate log file to avoid the `Permission denied` problem from section 2.5. The result has 725 samples, equal to 0.73 seconds of CPU in a 20 second run. I read the report with `perf report --no-children --sort symbol`.

```
# To display the perf.data header info, please use --header/--header-only options.
#
#
# Total Lost Samples: 0
#
# Samples: 725  of event 'cycles:P'
# Event count (approx.): 744063391
#
# Overhead  Symbol                                     IPC   [IPC Coverage]
# ........  .........................................  ....................
#
     5.39%  [k] entry_SYSRETQ_unsafe_stack               -      -
            |
            |--2.12%--__GI___pthread_disable_asynccancel
            |          |
            |          |--0.94%--pthread_cond_wait@@GLIBC_2.3.2
            |          |          cb_pop
            |          |          processor_thread_func
            |          |          start_thread
            |          |          clone3
            |          |
            |           --0.56%--sensor_thread_func
            |                     start_thread
            |                     clone3
            |
```

Figure 7. Top of perf report --no-children

The top entry is `entry_SYSRETQ_unsafe_stack` in the kernel at 5.39 %. This is the code that returns a thread to user mode after a system call. The two paths into it that I could see are `pthread_cond_wait` inside `cb_pop` of the Processor (0.94 %) and `sensor_thread_func` (0.56 %). What surprised me is that no application function stands out. Before running it I expected `snprintf` or `strstr` to lead, but the CPU time goes to the system call mechanism, not to computation. With 725 samples every small percentage has a large error, and I have only analyzed the first lines of the report.

From strace and perf I have two concrete proposals. The first is to let the Processor merge the INFO line and the alert lines of one sample into a single log message (at most 256 bytes), and to stop the Alert thread from forwarding each alert to `log_mq`. Each sample then needs 3 sends and 3 receives instead of 5 and 5, and the Logger receives 1 message per sample instead of 3. The second is to remove the 100 ms timeouts in Alert and Logger and use a blocking `mq_receive` plus a stop message sent by main at shutdown. In 30 seconds, 72 `mq_timedreceive` calls timed out, and those were wake-ups that did nothing. The gain from the second proposal is smaller than the first. I have not implemented either.

7. Real-time behavior and jitter

7.1 Implementation

The Sensor thread is set up as follows. With `-s 1` it calls `sched_setscheduler(0, SCHED_FIFO, ...)` with priority 50 and prints a warning if the call fails (in that case the report label says SCHED_OTHER, so it never claims FIFO by mistake). I picked 50 because it is above every SCHED_OTHER thread while leaving 51 to 99 free for kernel threads or a watchdog, in case the system moves to PREEMPT_RT later. The thread sleeps with `clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, ...)`, and the wake-up time is accumulated from the previous target (not from the moment it woke), so the error does not build up. `mlockall(MCL_CURRENT | MCL_FUTURE)` is called in `main` before the threads are created to avoid page faults at run time.

Jitter is the difference between the actual wake-up time and the expected time, in microseconds. Because absolute times are used, the thread cannot wake early, and a negative value (if one ever appeared) is set to 0. Each `-i 50 -d 55` run collects 1100 samples and the array holds 5000. P99 uses the nearest-rank method. The histogram has four bins: 0-100, 100-500, 500-1000 and above 1000 microseconds.

These numbers were measured inside the ARM guest running under QEMU emulation (TCG) on an x86 machine, so they include the delay of the virtual timer and of the QEMU process on the host. That is why I do not compare them directly with numbers from a real board.

7.2 Without background load

The two figures below show a SCHED_OTHER run followed by a SCHED_FIFO run with `-i 50 -d 55`. I ran two such pairs at two different times, and the figures show the second pair.

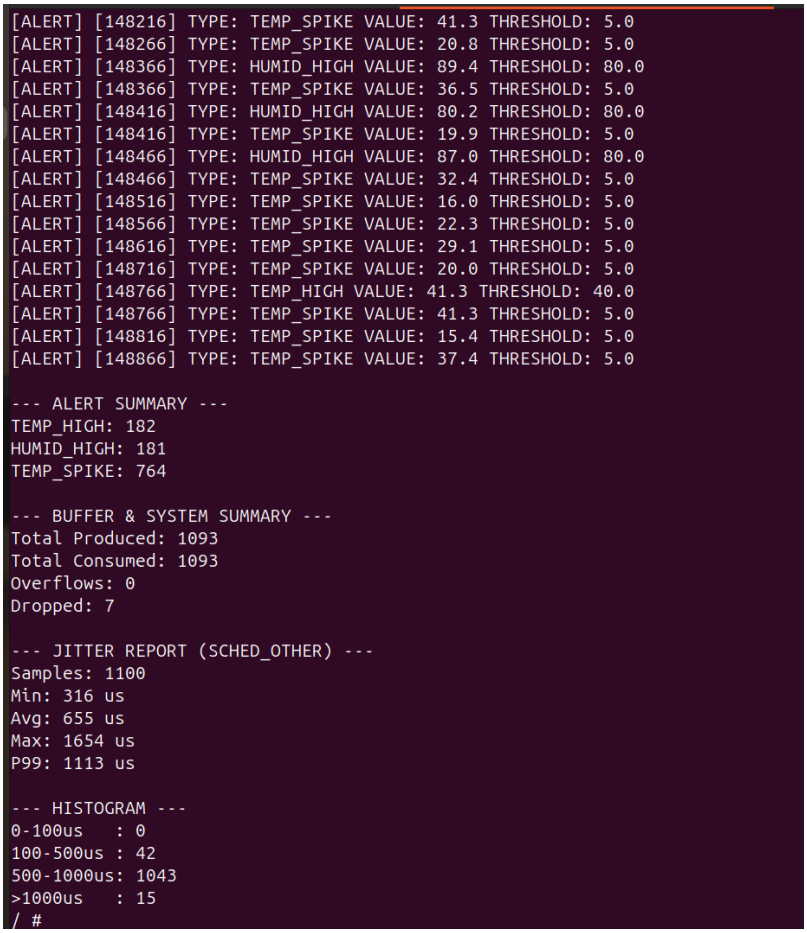


Figure 8. Run B, SCHED_OTHER, no load (tail of the output)

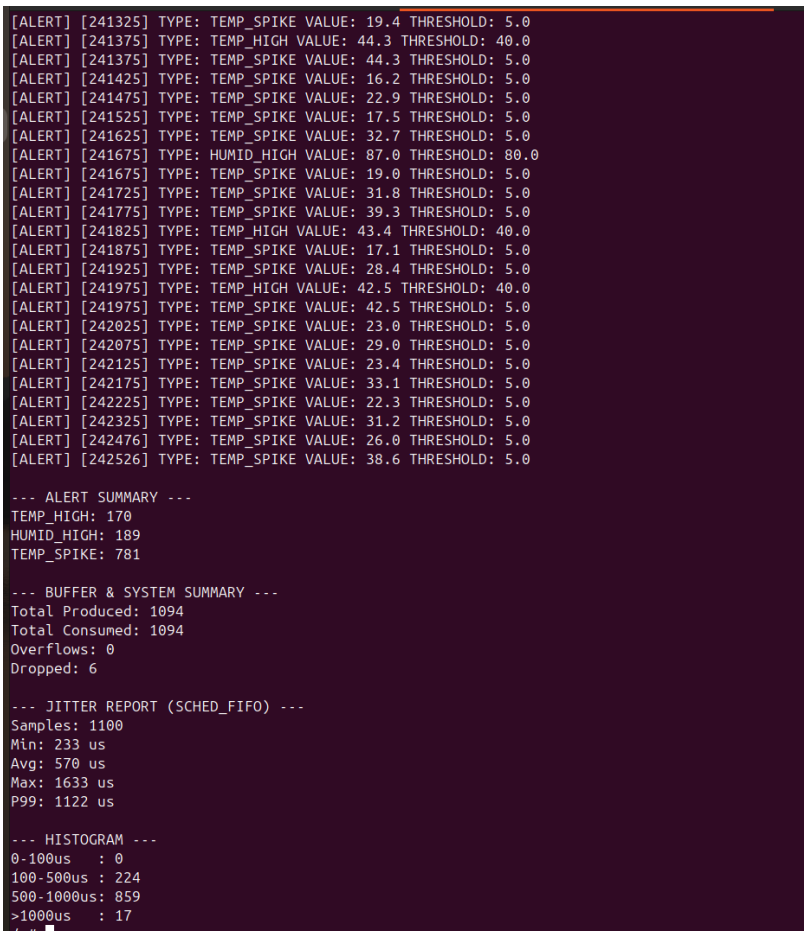


Figure 9. Run B, SCHED_FIFO 50, no load (tail of the output)

Table 13. Jitter without background load (50 ms, 1100 samples), in microseconds

	Min	Avg	Max	P99	0-100	100-500	500-1000	>1000
Run A, OTHER	161	564	4154	1440	0	267	792	41
Run A, FIFO	228	675	3352	1147	0	478	535	87
Run B, OTHER	316	655	1654	1113	0	42	1043	15
Run B, FIFO	233	570	1633	1122	0	224	859	17

The result is not clear. In run A, FIFO has a P99 20 % lower (1147 against 1440) but a higher average (675 against 564) and more samples above 1 ms (87 against 41). In run B, FIFO is slightly better on average (570 against 655) and the P99 values are almost equal (1122 against 1113). More important, the same OTHER mode run at two different times gives P99 from 1113 to 1440 and Min from 161 to 316, so the difference between two runs of one configuration is as large as the difference between the two modes. When the guest CPU is nearly idle, the normal scheduler already wakes the Sensor fast enough and SCHED_FIFO has little to improve.

7.3 With background load

To create CPU contention I started two busy loops, `while :; do :; done &` (the guest has 2 vCPUs), then ran OTHER and FIFO alternately, three runs of each, all with `-i 50 -d 55`.

Table 14. Jitter with two busy loops (50 ms, 1100 samples), in microseconds

Run	Mode	Min	Avg	Max	P99	>500 us
1	OTHER	171	343	1426	846	37
1	FIFO	133	214	448	330	0
2	OTHER	170	272	3318	535	15
2	FIFO	107	198	731	336	2
3	OTHER	172	296	1848	575	22
3	FIFO	91	192	1095	352	4
Median	OTHER	171	296	1848	575	22
Median	FIFO	107	198	731	336	2

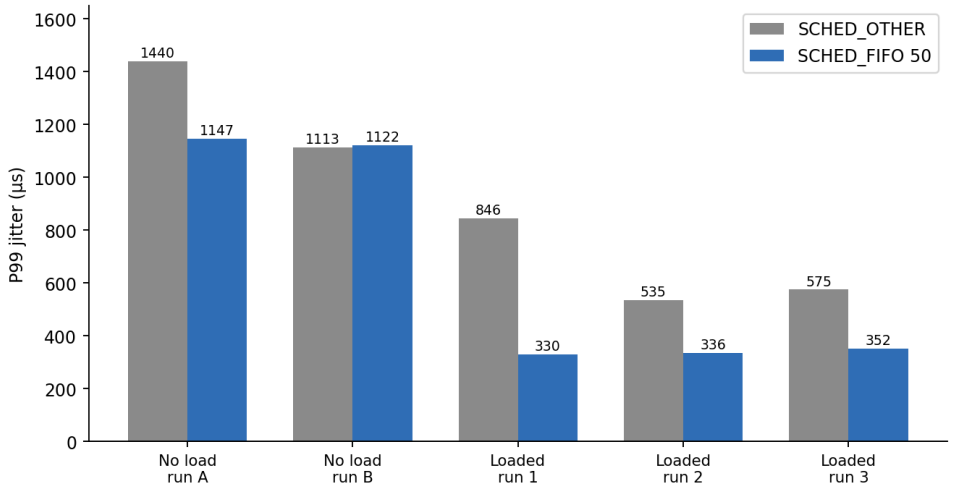


Figure 10. P99 jitter of each run

In all three runs FIFO has a lower P99 than OTHER, and the result is fairly stable (330, 336, 352 against 846, 535, 575). Using the medians, P99 drops from 575 to 336 microseconds, about 41.6 %. The average drops 33 % and Max drops 60 %. In runs 2 and 3 OTHER has a Max of 3318 and 1848, while FIFO never exceeds 1095.

There is one more run: FIFO with `-i 500 -d 60` and no load (120 samples), which gave Min 318, Avg 739, Max 2072 and P99 2031 microseconds, with 24 samples above 1 ms. That is worse than the loaded FIFO runs at 50 ms. My guess is that when the guest is idle the vCPU goes to sleep and QEMU wakes it more slowly, while a busy loop keeps the vCPU running. I have not tested this. A test would be to run `-i 500 -s 1` with background load and see whether P99 drops sharply. This run has only 120 samples, so it is not enough for statistics anyway.

7.4 Text histograms

The assignment asks for a text histogram with four bins. The two runs shown in Figures 8 and 9 look like this (one # stands for 25 samples):

```
SCHED_OTHER, run B (1100 samples)
    0-100 us :    0
  100-500 us :   42 ##
 500-1000 us : 1043 ########################################
  > 1000 us  :   15 #
```

```
SCHED_FIFO 50, run B (1100 samples)
    0-100 us :    0
  100-500 us :  224 #########
 500-1000 us :  859 ##################################
  > 1000 us  :   17 #
```

The next table gives the four bins for every 1100-sample run in this report.

Table 15. Histogram bins of all 50 ms runs (samples per bin)

Run	0-100 us	100-500 us	500-1000 us	>1000 us
No load A, OTHER	0	267	792	41
No load A, FIFO	0	478	535	87
No load B, OTHER	0	42	1043	15
No load B, FIFO	0	224	859	17
Loaded 1, OTHER	0	1063	32	5
Loaded 1, FIFO	0	1100	0	0
Loaded 2, OTHER	0	1085	11	4
Loaded 2, FIFO	0	1098	2	0
Loaded 3, OTHER	0	1078	21	1
Loaded 3, FIFO	1	1095	3	1

The histograms agree with the P99 table. With load, nearly all samples fall in the 100 to 500 microsecond bin for both modes, and OTHER has a longer tail. Without load, most samples sit in the 500 to 1000 microsecond bin in run B, which suggests a different floor from run A.

7.5 Answers to the assignment questions

Does SCHED_FIFO improve jitter? Yes, but under a condition. When the CPU is contended, P99 drops about 41.6 % (median of three runs) and all three runs point the same way. Without load the result is mixed and inside the variation between runs, so I do not claim an improvement. Each mode has only three loaded runs and two unloaded runs, so this is preliminary evidence and not a tight statistical result. The conclusion holds only for the conditions I tested: QEMU TCG, a 50 ms period and two busy loops as the load.

What is an acceptable maximum jitter? The assignment gives no number, so I chose 5 ms, which is 1 % of a 500 ms period. A temperature and humidity sensor in industrial use changes slowly, and a few milliseconds of offset in a read does not change the value. Every QEMU run I made has a Max below this limit (the largest is 4154 microseconds, SCHED_OTHER without load). If the limit is lowered to 1 ms, OTHER violates it in every run, and FIFO under load meets it in two of three runs. The assumption behind this is that absolute jitter does not depend on the period, since I measured at 50 ms and not at 500 ms.

What would reduce jitter further? My kernel is PREEMPT_NONE, which means code running in the kernel is not preempted, and that is a possible source of delay which SCHED_FIFO cannot fix. Options to try are PREEMPT_RT or at least PREEMPT, `isolcpus=1` with `taskset` to pin the Sensor to its own core and move the other threads to the remaining core (the guest has 2 vCPUs), interrupt affinity, and not printing alerts to the 115200 baud serial console during

measurement, because serial output generates interrupts and may slow things down. I have not tried any of these. On the host side, pinning the QEMU vCPU threads to fixed cores is also worth trying, because delay outside the guest cannot be fixed from inside it.

8. Summary of measurements

8.1 Samples, alerts and overflow

Table 16. Results of the QEMU runs ("-" means not recorded)

Run	Produced	Dropped	Overflow	TEMP_HIGH	HUMID_HIGH	TEMP_SPIKE
500 ms, 60 s, FIFO, no load	118	2	0	21	20	77
50 ms, 55 s, OTHER, run A	1083	17	0	208	187	759
50 ms, 55 s, FIFO, run A	1086	14	0	176	189	767
50 ms, 55 s, OTHER, run B (Figure 8)	1093	7	0	182	181	764
50 ms, 55 s, FIFO, run B (Figure 9)	1094	6	0	170	189	781
6 loaded runs (3 OTHER, 3 FIFO)	-	2, 1, 1 (OTHER) 0, 0, 1 (FIFO)	0	-	-	-

The loss rate (Dropped divided by the number of periods) is 1.7 % in the 500 ms run, 0.5 % to 1.5 % in the unloaded 50 ms runs, and 0 to 0.2 % in the loaded runs. The buffer overflow rate is 0 in every normal run. The only run with overflows (74) is the failed perf run from section 2.5, which is not in the table.

8.2 Why there are so many alerts

The driver produces temperature and humidity that are uniformly distributed and independent between reads, so the alert rates can be computed in advance. Temperature takes 301 values from 15.0 to 45.0, of which 50 are above 40.0, which is 16.6 %. Humidity takes 601 values and 100 are above 80.0, which is 16.6 %. For `TEMP_SPIKE`, two consecutive samples are independent, and the probability that they differ by more than 5.0 degrees is 62750 out of 90601 pairs, or 69.3 %. In run B with OTHER I measured 182, 181 and 764 out of 1093 samples, which is 16.6 %, 16.6 % and 69.9 %, very close to the theory. The other runs give a `TEMP_SPIKE` rate of 70 % to 71 %.

So the high alert count is a result of the simulation and not a fault. A real temperature sensor could hardly jump from 16 to 44 degrees between two reads 50 ms apart. I will say this in the demo video.

9. What is not verified yet

The items below are things I have not measured or settled. I list them so the reader does not misjudge the scope of the data.

- The alert was not observed after changing the variable in GDB (section 6.1). To close this I will rebuild with `make debug` and run: `gdb --args ./sms_app_x86_dbg -p`

`sms_mock_sensor -i 500 -d 15`, then `break processor_thread.c:46`, `run`, `info threads`, `set variable data.temperature=45.0`, `continue`, and check that a `[ALERT] ... TEMP_HIGH VALUE: 45.0` line is printed by the Alert thread, followed by `thread apply all bt`.

- Valgrind ran only with the one-line mock file, so the `TEMP_SPIKE` branch and the real driver read path were not checked under Valgrind.
- I have not counted each errno in the Sensor thread, so the cause of the 0.5 to 1.7 % of lost samples (`Dropped`) is unknown.
- Log rotation at 1 MB was not tested with a large file.
- I did not measure user-mode time separately, so the user versus kernel split is only estimated from strace and perf. The perf report was analyzed only at its top lines.
- The separate effect of `PTHREAD_PRIO_INHERIT` has no isolated experiment.
- The guesses about virtual timer delay in QEMU and about the vCPU sleeping when the guest is idle are untested.
- If `pthread_create` fails halfway, `main` returns without joining the threads that were already created. A fix would be to set the three stop flags, call `cb_shutdown`, and join only the threads that were created, before releasing resources. I have not implemented this.
- SIGINT was tested on the x86 build (under Valgrind). On QEMU the runs ended by `-d`. The `rmmod` and `dmesg` check after a run, which should show no oops, will be done while recording the demo video.

10. Looking back at the course

The hardest part for me was finding a bug when the program keeps running and reports nothing. The `mq_timedreceive` bug is the clearest example. After writing the four threads I ran them on QEMU and saw an ALERT SUMMARY full of zeros, while 263 random samples had passed through the Processor and `Overflows` reached 820. The buffer only overflows when the Processor is slow, so the cause had to be behind it. Running strace on the x86 build showed `mq_timedreceive` failing 14 times out of 14 calls. The `mq_receive` man page says the receive buffer must be at least `mq_msgsize`, and I had passed `MSG_SIZE - 1`.

Synchronization took time too, in a different way. At first I thought one mutex meant no deadlock was possible. When I listed what each thread waits for, I saw that the system can still hang without two locks: the Processor can block in `mq_send` on a queue whose reader, Alert or Logger, has already exited. That is why I switched to `mq_timedsend`, gave each thread its own stop flag, and joined in order.

If I started again I would change a few things. I would write the measurement scripts from the beginning instead of running each command by hand, since most of the bugs above showed up only when I could compare a number with one I had computed beforehand, such as 120 samples in 60 seconds or the 17 % chance of exceeding a threshold. I would make the driver wait for the remaining time when `read()` is blocking instead of returning `-EAGAIN`, because the application now has to retry and still loses a few samples per run. I would make the thread stacks smaller with `pthread_attr_setstacksize`, because four 8 MB stacks combined with `mlockall` give

an RSS of almost 33 MB for a program with about 450 KB of code. And I would use a stop message through the queue instead of the 100 ms timeout in Alert and Logger.

The skill I expect to use most at work is reading the output of strace, Valgrind and perf and comparing it with numbers I can work out from the code. For example, 1500 `mq_timedsend` calls in 30 seconds match 5 sends per sample, and that is why I trust the other numbers in the table. The second skill is knowing when a result is not enough to conclude. Comparing SCHED_FIFO with SCHED_OTHER without load gave a mixed picture, and only after adding two busy loops did the difference become clear.

With more time I would do two things. The first is to make the driver generate data as a random walk (each step at most 0.5 degrees) so that `TEMP_SPIKE` means something, since it now fires on about 70 % of samples. The second is to add a fifth thread that publishes samples and alerts to an MQTT broker. That thread would take data through a third queue, like the Logger, so a slow network does not block the Sensor or Processor.

References

- Assignment 02 handout, Embedded Linux Systems.
- Christopher Hallinan, Embedded Linux Primer, 2nd Edition, chapters 8 to 19.
- Corbet, Rubini, Kroah-Hartman, Linux Device Drivers, 3rd Edition (lwn.net/Kernel/LDD3).
- Michael Kerrisk, The Linux Programming Interface, chapters on threads.
- Man pages: pthread_create(3), mq_open(3), mq_receive(3), clock_nanosleep(2), sched_setscheduler(2), strace(1).
- Brendan Gregg, Systems Performance 2nd Edition, brendangregg.com/perf.html.
- PREEMPT_RT: wiki.linuxfoundation.org/realtime.

Appendix A. Commands used to produce the data

On the host, inside the `app` directory:

```
make host
echo "1500,42.3,85.2" > sms_mock_sensor
valgrind --leak-check=full --show-leak-kinds=all --track-fds=yes \
  ./sms_app_x86 -p sms_mock_sensor -i 200 -d 60
strace -f -c -o ../logs/strace_sms.txt ./sms_app_x86 -p sms_mock_sensor -i 100 -d
30
sudo perf record -F 999 -g -o /tmp/perf.data ./sms_app_x86 -p sms_mock_sensor \
  -l /tmp/sms_perf.log -i 10 -d 20
sudo perf report -i /tmp/perf.data --no-children --sort symbol --stdio
gdb --args ./sms_app_x86 -p sms_mock_sensor -i 500 -d 15
```

Inside QEMU:

```
insmod sms_sensor_driver.ko
cat /proc/sms_stats
./sms_app -i 500 -d 60 -s 1        # read /proc/PID/status and maps while it runs
cat /proc/sms_stats
./sms_app -i 50 -d 55 -s 0         # SCHED_OTHER
```

```
./sms_app -i 50 -d 55 -s 1         # SCHED_FIFO
while :; do :; done &              # start twice for background load
```

Appendix B. Mapping to the assignment requirements

Table 17. Requirements and where they are covered

Requirement	Section
Driver: CSV read, /proc, ioctl	2.2, 4.1, 4.2
Four threads, buffer, queues	2.1, 2.3
No deadlock, no race	3.1 to 3.4
Clean shutdown on SIGINT, exit code	2.4, 3.3
60 second stable run, all threads active	4.2
Valgrind, memory footprint, maps	5.1, 5.2
Driver integration, error handling, read_count	4.2, 4.3
GDB	6.1 (alert observation still open, see section 9)
strace, perf	6.2, 6.3
SCHED_FIFO, jitter, comparison, proposals	7
Text histogram	7.4
Summary table of measurements	8
Architecture diagram, state machine, design decisions	2.1, 2.4, 2.5
Reflection	10

Appendix C. Evidence files

The numbers in this report come from the following files, which are also part of the submitted package.

Table 18. Evidence files

File	Used for
logs/valgrind_sms.txt	Section 5.1, heap summary, file descriptors
logs/strace_sms.txt	Section 6.2, syscall counts
logs/perf_sms.txt	Section 6.3, perf report
logs/proc_sms_stats_before.txt, proc_sms_stats_after.txt	Section 4.2, read_count before and after
logs/jitter_report.txt	Section 7, jitter reports of all 50 ms runs
logs/sms_sample_run.txt, sms.log	Terminal log of a demo run and the log file written by the Logger
logs/qemu_run.txt	Full QEMU console output of the runs in sections 4, 5.2, 7 and 8